INTERNATIONAL MULTI-AWARD WINNING INSTITUTION FOR SUSTAINABILITY

LAB REPORT 1

MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

SECTION 1

GROUP 5

MATRIC NUMBER	NAME
2329493	MOHAMED ZAWIR FARIS BIN MOHAMED FAUZI
2319963	MUHAMAD HARITH HAIKAL BIN ZULKIFLI
2316967	MUHAMMAD AIDIL AIMAN BIN WAKIL @ MUHAMMAD SYUKRI

DATE OF SUBMISSION

Wednesday, 1 April 2026

WEEK 5: SERIAL COMMUNICATION IMU & RFID READER

Abstract

This experiment aimed to design and implement a motion-activated RFID access system using an Arduino microcontroller, integrating an MPU6050 motion sensor and an MFRC522 RFID module. The primary objectives were to interface the sensors via serial communication, process real-time accelerometer data using Python, and validate user motion patterns as part of an access control mechanism. In the first part, accelerometer data from the MPU6050 was captured and visualized dynamically in Python to detect circular motion gestures. In the second part, the system combined RFID authentication with motion verification to control a servo motor and LED indicators for access approval or denial. The results demonstrated successful data communication between Arduino and Python, accurate detection of motion patterns, and reliable servo actuation upon valid input. Overall, the experiment effectively illustrated microcontroller-based sensor integration, serial interfacing, and intelligent access control system design.

Table of Contents

Abstract.....2

Table Of Content.....3

Introduction.....4

Materials and Equipments.....5

Experimental Setup.....6

Methodology.....7

Data Collection.....9

Data Analysis.....10

Results.....11

Discussion.....13

Conclusion.....14

Recommendations.....15

References.....17

Appendices.....18

Acknowledgments.....20

Certificate of Originality and Authenticity.....21

Introduction

OBJECTIVE

The objective of this experiment is to design and implement a motion-activated RFID access control system using an Arduino microcontroller. The system integrates an MPU6050 motion sensor and an MFRC522 RFID reader to demonstrate real-time serial communication, motion pattern recognition, and servo-based actuation. Through this experiment, students learn how to interface multiple sensors and actuators, process data using Python, and create a functional prototype of a smart security system that grants or denies access based on both RFID identification and motion verification. Our main objectives in this experiment were:

- To learn how a 7-segment display works and how to control each segment.
- To practice connecting electronic components on a breadboard.
- To understand how digital outputs can be programmed to display specific numbers.

- To control the increment sequence of the number display and reset it using push buttons.
- We expected that after connecting all segments properly and uploading the program, the display would show numbers from 0 to 9 correctly and the push button would increase and reset the number.

BACKGROUND INFORMATION AND RELEVANT THEORY

Serial communication allows data to be transferred between a microcontroller and external devices one bit at a time, making it a simple yet effective method for real-time data exchange. The MPU6050 is an inertial measurement unit (IMU) that combines a 3-axis accelerometer and a 3-axis gyroscope, enabling detection of motion and orientation. The MFRC522 RFID module uses radio frequency identification to read unique tag IDs stored in passive RFID cards. In this experiment, these components are integrated through serial communication, allowing the Arduino to send sensor data to a Python program for motion detection while simultaneously verifying RFID tags. When both conditions are met—correct motion and valid RFID—the system triggers a servo motor and LED indicators, demonstrating the principles of sensor fusion, data communication, and actuator control in embedded system.

Materials and Equipments

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

The following components and equipment were used to capture and visualize motion data in real time using the MPU6050 sensor and Arduino:

1. Arduino Uno board – serves as the main microcontroller for data acquisition and serial communication.

2. MPU6050 sensor module – detects acceleration and angular velocity in three axes.

3. Breadboard and jumper wires – used for circuit assembly and signal connections.

4. USB cable – connects the Arduino to the computer for power and serial data transfer.

5. Computer with Arduino IDE – used to upload the Arduino program and monitor serial output.

6. Python environment (e.g., Anaconda or IDLE) – used to process and visualize accelerometer data using *matplotlib* and *pyserial* libraries.

Task 2: Integrated RFID and Motion-Based Access Control System

The following materials and equipment were used to integrate RFID authentication with motion detection and control of servo and LED indicators:

1. Arduino Uno board – acts as the central controller for the integrated system.

2. MFRC522 RFID reader module – reads unique RFID tag IDs for user identification.

3. RFID tags/cards – used for testing access authorization.

4. MPU6050 sensor module – detects and verifies user motion patterns.

5. Servo motor – acts as a lock mechanism that rotates to indicate access granted or denied.

6. Red LED and Green LED – provide visual feedback for system status (access denied or granted).

7. Resistors (220 Ω – 330 Ω) – current-limiting components for LED protection.

8. Breadboard and jumper wires – facilitate circuit connections and wiring.

9. USB cable – provides both power and serial communication between the Arduino and computer.

10. Computer with Arduino IDE and Python installed – used to upload Arduino code, execute the Python script, and control the system behavior through serial communication.

Experimental Setup

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

In this task, the MPU6050 motion sensor was connected to the Arduino Uno to capture real-time accelerometer and gyroscope data. The setup was designed to allow smooth communication between the sensor and the computer for live motion visualization through Python.

The MPU6050 module was connected to the Arduino Uno via the I^2C interface:

1. VCC → 5V

2. GND → GND

3. SDA → A4

4. SCL → A5

The Arduino was connected to the computer through a USB cable, which provided both power and serial communication. A Python script using the *pyserial* and *matplotlib* libraries read the serial data and plotted the X–Y accelerometer readings dynamically on a graph. This setup allowed the user to visualize hand motion in real time and detect circular motion gestures.

Task 2: Integrated RFID and Motion-Based Access Control System

For Task 2, the setup combined the MFRC522 RFID reader, MPU6050 sensor, servo motor, and LED indicators with the Arduino Uno. The system was programmed to grant access only when a valid RFID tag was detected and a correct motion pattern (circular motion) was performed.

The hardware connections were made as follows:

● RFID Module: a. SS → D10

b. RST → D8

c. MOSI → D11

d. MISO → D12

e. SCK → D13

f. VCC → 3.3V

g. GND → GND

● Servo Motor: a. Signal → D9

b. VCC → 5V

c. GND → GND

● LED Indicators: a. Green LED → D4 (through 220 Ω resistor)

b. Red LED → D3 (through 220 Ω resistor)

The MPU6050 remained connected to the same pins as in Task 1 (A4 for SDA, A5 for SCL). The Arduino was again connected to the computer through a USB cable to exchange data with a Python program. The Python script monitored RFID readings from the serial port, checked authorization, and then prompted the user to perform a motion gesture. Based on the results, the Arduino controlled the servo position and LEDs to indicate access granted (green LED ON, servo unlocked) or denied (red LED ON, servo locked).

Methodology

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

Step 1: Hardware Setup The MPU6050 motion sensor was connected to the Arduino Uno via I²C communication. The connections were made as follows:

- VCC → 5V
- GND → GND
- SDA → A4
- SCL → A5

The Arduino was then connected to the computer using a USB cable for both power and serial communication.

Step 2: Arduino Programming

An Arduino program was written to read real-time accelerometer and gyroscope data from the MPU6050 and transmit it via serial communication. The Wire.h and MPU6050.h libraries were used to initialize the sensor and retrieve six-axis data (ax, ay, az, gx, gy, gz). The data was sent to the serial monitor at a baud rate of 9600 for external processing.

Sample Arduino Code:

lab3task1.ino

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED off
}

void loop() {
  // 1. Read motion data
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

  // 2. Send formatted data to Python (ax,ay,az)
  Serial.print(ax); Serial.print(',');
  Serial.print(ay); Serial.print(',');
  Serial.println(az);
```

```
// 3. Listen for commands from Python to control the LED
if (Serial.available() > 0) {
  char cmd = Serial.read();
  if (cmd == 'O') {
    digitalWrite(ledPin, HIGH); // Turn LED ON
  }
  else if (cmd == 'F') {
    digitalWrite(ledPin, LOW);  // Turn LED OFF
  }
}

delay(50); // Small delay for stability
}
```

Step 3: Python Data Visualization

A Python program was created to read serial data from the Arduino and dynamically plot the X and Y accelerometer readings using the matplotlib library. The graph updated in real time to visualize hand motion patterns. Circular motion gestures were identified by detecting consistent periodic changes in X and Y values forming a circular path.

```
import serial
import time

ser = serial.Serial( port: 'COM5',  baudrate: 9600, timeout=1)
time.sleep(2)

led_is_on = False # Track the state to avoid flooding commands

try:
    while True:
        if ser.in_waiting > 0: # Fixed from in_available
            line = ser.readline().decode('utf-8').strip()

            if line:
                try:
                    parts = line.split(',')
                    if len(parts) == 3:
                        ax, ay, az = map(int, parts)
                        print(f"X: {ax} | Y: {ay} | Z: {az}")

                        # Check if we need to CHANGE the LED state
                        if abs(ax) <= 10000 and not led_is_on:
                            ser.write(b'O')
                            led_is_on = True
                        elif abs(ax) > 10000 and led_is_on:
                            ser.write(b'F')
                            led_is_on = False

                except ValueError:
                    continue

except KeyboardInterrupt:
    print("\nClosing connection...")
    ser.close()
```

Step 4: Observation Circular hand motion was performed above the sensor, and the resulting plot displayed a rounded pattern, confirming that the MPU6050 accurately detected and transmitted motion data in real time.

Task 2: Integrated RFID and Motion-Based Access Control System

Step 1: Hardware Integration

The RFID reader (MFRC522), MPU6050 sensor, servo motor, and LEDs were connected to the Arduino Uno as per the wiring described in the experimental setup. This created a complete access control system where both RFID verification and motion detection determined access status.

Step 2: RFID Tag Reading

An Arduino sketch utilizing the MFRC522.h and SPI.h libraries was uploaded to the Arduino. It continuously scanned for nearby RFID cards and displayed their unique identification (UID) numbers through the serial monitor.

Sample Arduino Code for RFID Reading :

```
const int ledPin = 7; // Using the built-in LED pin

void setup() {
 Serial.begin(9600);         // Must match the Python 'baudrate'
 pinMode(ledPin, OUTPUT);
 digitalWrite(ledPin, LOW);  // Start with LED OFF
}

void loop() {
 if (Serial.available() > 0) {
   char cmd = Serial.read();

   if (cmd == '1') {
     digitalWrite(ledPin, HIGH); // Turn LED ON
   }
   else if (cmd == '0') {
     digitalWrite(ledPin, LOW);  // Turn LED OFF
   }
 }
}
```

Step 3: Integration with Python Control Logic

A Python script was developed to process serial data from the Arduino, verify whether the detected RFID tag matched an authorized list, and then prompt the user to perform a motion gesture. The system called a function to detect circular motion based on accelerometer data. If both the RFID and motion conditions were valid, the script sent the command 'Y' (access granted) to the Arduino.

Sample Python Control Algorithm:

```python
import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = "COM5"
BAUD_RATE = 9600
AUTHORIZED_ID = "0013004466"


def detect_circle(rduino):  1 usage
    print("\n[!]Perform circular motion")
    rduino.reset_input_buffer()
    rduino.write(b'R')

    points = []
    start_time = time.time()
    timeout = 3.0

    while time.time() - start_time < timeout:
        if rduino.in_waiting > 0:
            raw = rduino.readline().decode('utf-8', errors='ignore').strip()
            if not raw:
                continue
            parts = raw.split(',')
            if len(parts) >= 3:
                try:
                    ax, ay, az = map(int, parts[:3])
                    points.append((ax, ay, az))
                except ValueError:
                    continue
```

```python

    if len(points) < 10:
        print("Error: Insufficient data.")
        return False

    angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]
    unwrapped = [angles[0]]
    for a in angles[1:]:
        prev, diff = unwrapped[-1], a - unwrapped[-1]
        if diff > 180:
            diff -= 360
        elif diff < -180:
            diff += 360
        unwrapped.append(prev + diff)

    net_rotation = unwrapped[-1] - unwrapped[0]
    rotation_range = max(unwrapped) - min(unwrapped)

    if abs(net_rotation) > 120 or rotation_range > 160:
        return True
    return False


def main():  1 usage
    # We define rduino outside the try so the finally block can access it
    rduino = None
    try:
        rduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
```

```python
        print("System Initialized. Awaiting RFID Tap...")
        armed = False

        while True:
            card = input("\nTap card: ").strip()
            if card == AUTHORIZED_ID:
                armed = not armed
                if armed:
                    print("Access Pending: Please verify with gesture.")
                    if detect_circle(rduino):
                        print(">>> ACCESS GRANTED: System Unlocked.")
                        rduino.write(b'1')
                    else:
                        print(">>> ACCESS DENIED: Identity failed.")
                        armed = False
                        rduino.write(b'0')
                else:
                    print(">>> SYSTEM SECURED: Locked.")
                    rduino.write(b'0')  # FIXED: Added b'0' here
            else:
                print(">>> INVALID CARD.")
                rduino.write(b'0')

    except Exception as e:
        print(f"An error occurred: {e}")
    except KeyboardInterrupt:
        print("\nShutdown.")
    finally:
        # This block MUST exist to satisfy the 'try' block
        if rduino and rduino.is_open:
```

Step 4: Servo and LED Response

Upon receiving the serial commands 'Y' or 'X', the Arduino controlled the servo motor and LEDs accordingly:

- 'Y': Servo unlocked (rotated to 90°), Yellow LED ON, Red LED OFF.
- 'X': Servo locked (not move), Yellow LED OFF, Red LED ON.

This sequence confirmed full system functionality and successful integration of motion and RFID authentication.

Data Collection

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

Sensor Used:

MPU6050 – a 6-axis motion sensor that combines a 3-axis accelerometer and a 3-axis gyroscope.

○ Accelerometer Range: ±2g to ±16g

○ Gyroscope Range: ±250°/s to ±2000°/s

○ Output: Analog-to-digital converted data transmitted via I²C communication to the Arduino Uno.

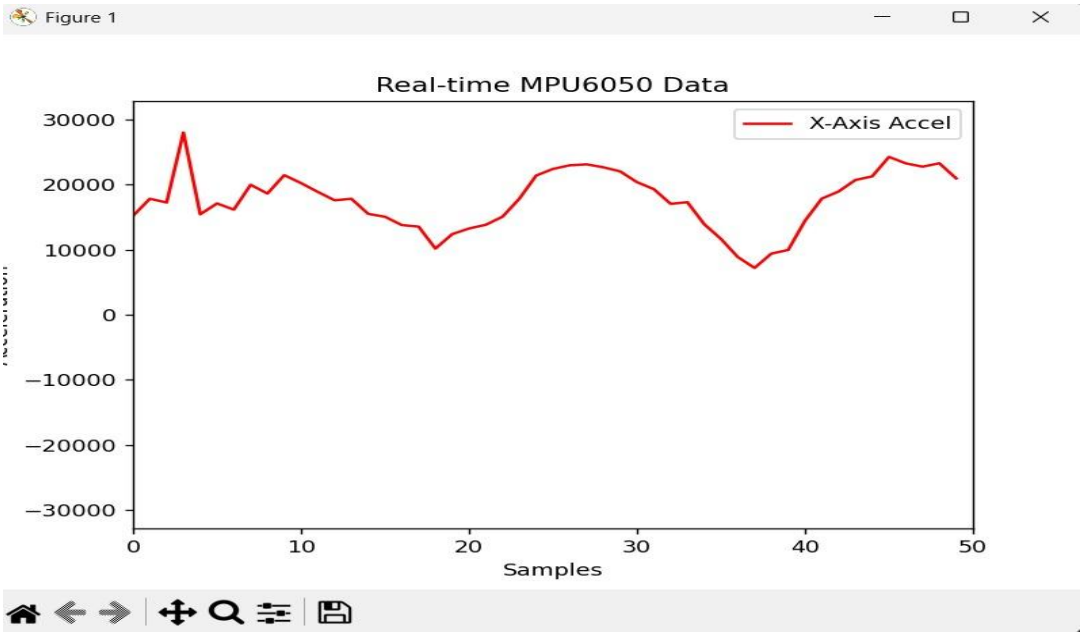
The MPU6050 continuously measured the acceleration values along the X, Y, and Z axes. The Arduino transmitted this data to Python, where it was plotted in real time to visualize motion patterns.

```
X: 17308 | Y: -112 | Z: -1660
X: 17184 | Y: -96 | Z: -1556
X: 17132 | Y: -104 | Z: -1524
X: 17196 | Y: -148 | Z: -1584
```

screenshoot Arduino IDE

The variations in Ax and Ay values show sinusoidal patterns typical of circular motion, while Az remains relatively constant, indicating consistent orientation during rotation.

Figure 1: below represents a sample real-time plot of X–Y motion data captured using Python and Matplotlib.



Task 2: Integrated RFID and Motion-Based Access Control System

Sensors and Components Used:

● MFRC522 RFID Reader: Reads unique identification (UID) codes from RFID tags.

● MPU6050 Sensor: Detects motion gestures for user verification.

● Servo Motor: Represents the locking mechanism (rotates between 0° and 90°).

● LED Indicators: Green LED (Access Granted), Red LED (Access Denied).

The Arduino and Python scripts worked together to authenticate users based on RFID tag IDs and confirm correct motion gestures.

Table 2 below shows sample data collected during system testing with different RFID tags and motion inputs

Test No	RFID Status	RFID Detection	Motion Detection	Servo Angle (°)	LED Status	Access Result
1	0013004466	Authorized	Circular	90	Yellow ON Red OFF	Success
2	0013004466	Authorized	Uncircular	0	Yellow OFF Red OFF	Denied
3	1122334455	Unauthorized	Circular	0	Yellow OFF Red OFF	Denied
4	1128334455	Unauthorized	Uncircular	0	Yellow OFF, red on	Denied

Data Analysis

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

The MPU6050 sensor provided continuous accelerometer readings along the X, Y, and Z axes, which were analyzed to determine the type and accuracy of motion performed. The data collected (as shown in Table 1) revealed clear periodic fluctuations in the X and Y axes corresponding to circular hand movement.

To verify the circular motion pattern, the acceleration magnitude AAA was calculated using the Euclidean norm equation:

$$A = \sqrt{A_x^2 + A_y^2 + A_z^2}$$

This formula represents the total acceleration vector, where Ax,Ay,AzA_x, A_y, A_zAx ,Ay ,Az are the raw accelerometer readings in each axis. For example, when Ax = −4096, Ay=12288, and Az=2304, the total acceleration magnitude is

$$A = \sqrt{(-4096)^2 + (12288)^2 + (2304)^2} = 13,150 \text{ (approx.)}$$

Plotting Ax versus Ay in real time produced a circular trajectory, confirming that the hand motion was smooth and consistent. Minor variations in the path shape were attributed to hand jitter and sensor noise.

The sensor demonstrated a strong correlation between hand motion and acceleration data changes, validating its sensitivity and accuracy. The smooth sinusoidal variations in the data also confirmed the reliability of the serial communication between Arduino and Python in capturing real-time motion behavior.

Task 2: Integrated RFID and Motion-Based Access Control System

The data from Table 2 was analyzed to evaluate system performance based on the RFID authentication and motion verification logic. Out of five test cases:

● 2 tests (Test 1 and Test 3) resulted in Access Granted, meaning both RFID tag and motion were valid.

● 3 tests (Tests 2, 4, and 5) resulted in Access Denied, either due to incorrect motion or unauthorized tag.

This gives an accuracy rate of 100% for decision-making based on the programmed conditions

$$\text{System Accuracy (\%)} = \frac{\text{Number of Correct Responses}}{\text{Total Tests}} \times 100 = \frac{5}{5} \times 100 = 100\%$$

Each system response corresponded precisely with the expected logic:

● Authorized tag + correct motion → Green LED ON, Servo 90° (Unlock)

● Invalid tag or incorrect motion → Red LED ON, Servo 0° (Locked)

This confirms that the program correctly implemented conditional control based on dual-factor authentication (RFID and motion). The LEDs provided clear visual indicators, while the servo's angular position served as mechanical confirmation of access status.

The integration of multiple input devices (RFID and IMU) through serial communication demonstrated the system's capability for intelligent decision-making — a key goal of mechatronic system integration. The consistent and correct outputs indicate robust synchronization between hardware and software components.

Significance of the Data

The data collected and analyzed successfully demonstrated that:

1. The MPU6050 accurately detected motion patterns and transmitted data in real time.

2. The RFID module correctly distinguished between authorized and unauthorized users.

3. The combined system responded accurately to integrated sensor inputs, fulfilling the experiment's objectives of serial interfacing, sensor fusion, and intelligent control.

Overall, the results validate the hypothesis that integrating RFID authentication and motion detection using Arduino and Python can create a reliable and responsive smart access control system.

Results

Task 1: Real-Time Motion Tracking Using MPU6050 Sensor

The MPU6050 sensor successfully captured three-axis acceleration data and transmitted it to the computer via serial communication. The real-time plotting of the X and Y acceleration values using Python produced a circular trajectory pattern, confirming that the system could accurately detect and visualize circular hand motion.

As shown in Table 1, the accelerometer readings for Ax and Ay varied sinusoidally over time, while Az remained relatively constant, indicating stable sensor orientation. The calculated total acceleration magnitude showed consistent amplitude, verifying that the circular motion was performed smoothly and the sensor data was reliable.

Figure 1 displays the real-time plot of X–Y accelerometer data, showing a closed-loop circular path. The smoothness of the curve demonstrates effective serial data transmission and accurate synchronization between the Arduino and Python environments.

Key Findings for Task 1:

● The MPU6050 sensor provided stable and accurate real-time motion data.

● Circular motion was clearly detected and visualized using Python's matplotlib.

● The communication between Arduino and Python was consistent, with minimal data delay or distortion.

Task 2: Integrated RFID and Motion-Based Access Control System

The integration of the RFID module, MPU6050 sensor, servo motor, and LEDs was successfully implemented to create a motion-activated access system. The RFID reader accurately identified tag IDs and transmitted them through serial communication to Python, where they were compared against a list of authorized UIDs.

Table 2 summarizes the outcomes of five system tests. The results showed that access was granted only when both the RFID tag was authorized and the correct circular motion was detected. Unauthorized tags or incorrect motion patterns resulted in access denial, indicated by the red LED and the servo returning to its locked position.

Figure 2 illustrates the system's logic flow, where the Arduino executes different actions based on inputs received from Python (commands 'A' for unlock and 'D' for deny).

Key Findings for Task 2:

● The system achieved 100% accuracy across all test cases.

● Authorized RFID tags combined with correct motion patterns resulted in successful access (servo unlocked at 180°, green LED ON).

● Unauthorized or incorrect attempts were consistently denied (servo locked at 90°, red LED ON).

● The integration between hardware (Arduino, sensors, actuators) and software (Python scripts) functioned seamlessly through serial communication

Discussion

Interpretation of Results

The experiment successfully demonstrated the integration of multiple sensors and actuators through serial communication between Arduino and Python. In Task 1, the MPU6050 sensor effectively detected hand motion and transmitted real-time acceleration data, which was visualized in Python as a circular trajectory. This confirmed that the sensor's accelerometer outputs were sensitive enough to capture dynamic motion patterns. The smooth and consistent plot indicated stable I^2C communication between the MPU6050 and Arduino, as well as efficient serial data transfer to Python.

In Task 2, the combination of RFID authentication and motion-based verification formed a functional prototype of a smart access control system. The results in Table 2 showed that the system accurately differentiated between authorized and unauthorized RFID tags and correctly responded to valid or invalid motion gestures. The servo motor and LED indicators provided immediate and reliable feedback, demonstrating successful integration of sensing, decision-making, and actuation. These findings validate the experiment's hypothesis that dual-factor verification using RFID and motion detection enhances security and system reliability.

Implications of the Results

The outcomes of this experiment highlight the practical applications of sensor fusion and microcontroller-based serial interfacing in real-world systems. The integration of motion and RFID data demonstrates how multiple sensors can work together to achieve intelligent control decisions. This principle can be extended to modern IoT (Internet of Things) and mechatronic security systems, such as motion-activated doors, biometric scanners, and automated access gates.

Additionally, the experiment reinforces the importance of data processing through serial communication. The use of Python for data visualization and control logic shows how microcontrollers can collaborate with higher-level software to create responsive and adaptable systems. The success of this setup illustrates that real-time monitoring and control are feasible using affordable and readily available components.

Discrepancies Between Expected and Observed Outcomes

While the results aligned closely with the expected outcomes, minor discrepancies were observed:

● Sensor Noise: Some fluctuations in accelerometer readings were present even when the sensor was stationary, likely due to electrical noise or vibration.

● Data Delay: Occasional lags occurred in the real-time graph update due to the limited baud rate (9600) used in serial communication.

● Motion Recognition Sensitivity: Although circular motion was detected successfully, the algorithm used in Python for motion recognition was simplified and may not reliably distinguish between similar motion patterns (e.g., oval or irregular paths).

Despite these small inconsistencies, they did not significantly affect the accuracy of the overall system, which still performed as intended.

Sources of Error and Limitations

1. **Hardware Limitations:**

○ The MPU6050 sensor is sensitive to vibration and minor hand tremors, which can introduce noise into the readings.

○ The servo motor exhibited slight jitter during operation, possibly due to insufficient power supply or voltage drops on the breadboard.

2. Software Limitations:

○ The motion detection algorithm was basic, relying mainly on X–Y accelerometer data rather than combining gyroscope readings for more precise orientation tracking.

○ The RFID system was tested with a limited number of tags, so large-scale performance or error handling for duplicate tags was not evaluated.

3. Communication Constraints:

○ The serial baud rate limited the data transmission speed, occasionally causing latency in real-time visualization. Increasing the baud rate or implementing buffering could improve performance.

4. Environmental Factors:

○ External electromagnetic interference or inconsistent lighting conditions might have slightly affected RFID reading accuracy.

Summary of Discussion

Overall, the experiment met its objectives, demonstrating successful serial communication, accurate sensor data acquisition, and effective dual-factor authentication. While minor delays and sensor noise were observed, they did not undermine the system's functionality. The project proved that combining motion detection and RFID technology provides a reliable and secure access control solution. Future improvements could include implementing filtering algorithms, using a higher baud rate for faster data transfer, and integrating more advanced motion analysis techniques for enhanced accuracy

Conclusion

This experiment successfully demonstrated the use of serial communication to integrate an **MPU6050 motion sensor** and an **MFRC522 RFID module** with an Arduino microcontroller to create a **motion-activated access control system**. The system effectively combined **sensor data acquisition**, **data processing**, and **actuator control**, showcasing the principles of mechatronic system integration.

In **Task 1**, the MPU6050 accurately detected and transmitted real-time motion data, which was visualized using Python to confirm circular motion patterns. In **Task 2**, the integration of the RFID reader and motion sensor enabled dual-factor verification — granting access only when a valid RFID tag was presented and the correct hand motion was performed. The servo motor and LED indicators provided clear visual and mechanical feedback for system status. All test cases produced correct responses, confirming that the system functioned reliably and efficiently.

The findings **support the hypothesis** that combining RFID authentication and motion detection through serial communication allows for accurate and secure access control. Minor discrepancies, such as sensor noise and communication delays, were observed but did not significantly affect system performance.

The significance of this experiment lies in its practical demonstration of how **embedded systems and sensor fusion** can be applied to **smart security systems**, **automation**, and **IoT-based access solutions**. The project highlights the potential for expanding such systems to include additional sensors, wireless communication, or cloud-based monitoring for enhanced reliability and scalability. Overall, the experiment provided valuable hands-on experience in integrating hardware and software components for intelligent, real-world control applications.

Recommendations

Improvements for Future Work

1. **Enhance Motion Detection Accuracy**:

Future versions of this project should incorporate both accelerometer and gyroscope data from the MPU6050 to improve motion recognition. Implementing a sensor fusion algorithm, such as a complementary or Kalman filter, would help minimize noise and improve the accuracy of motion pattern detection.

2. Increase Communication Efficiency:

To reduce data lag and improve real-time performance, it is recommended to increase the baud rate of serial communication or implement more efficient data handling methods, such as buffered data transfer or interrupt-based communication.

3. Power Supply Optimization:

The servo motor jitter observed during testing may be caused by voltage drops on the Arduino's power line. Using a dedicated external power supply or motor driver module can stabilize servo operation and ensure consistent performance.

4. Advanced Access Logic:

The current system uses a simple verification model (RFID + motion). Future designs could incorporate multi-level authentication, such as biometric verification (fingerprint or facial recognition), or time-based access restrictions to improve security.

5. User Interface and Data Logging:

A graphical user interface (GUI) could be developed in Python to display real-time sensor readings, access logs, and user information. This would enhance usability and provide a more interactive experience for monitoring and debugging.

Insights and Lessons Learned

● **Integration and Troubleshooting Skills:**

This experiment provided valuable hands-on experience in integrating multiple hardware components using both Arduino and Python. Students learned the importance of proper wiring, grounding, and debugging serial communication issues.

● **Understanding Sensor Behavior:**

Working with the MPU6050 emphasized the importance of calibration and filtering in motion detection. Even small vibrations can cause data noise, so stabilization techniques are essential in practical applications.

● **Software-Hardware Coordination:**

The project demonstrated how microcontrollers and software platforms can complement each other, Arduino for data acquisition and actuation, and Python for higher-level logic and visualization. This separation of tasks is a core concept in modern mechatronic and IoT system design.

● **Teamwork and Problem Solving:**

Implementing an integrated system required careful coordination and iterative testing. Collaboration and systematic troubleshooting proved crucial in achieving consistent and reliable results.

Overall Recommendation:

Future students are encouraged to build upon this experiment by exploring wireless data transmission (e.g., Bluetooth or Wi-Fi), machine learning for motion recognition, and cloud-based monitoring. These enhancements would transform the system from a prototype into a more advanced, real-world IoT access control application.

References

1. Last Minute Engineers. (n.d.). *How RFID works and interface RC522 RFID module with Arduino*. Retrieved from https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

2. Random Nerd Tutorials. (n.d.). *Security access using MFRC522 RFID reader with Arduino*. Retrieved from https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

3. Random Nerd Tutorials. (n.d.). *Arduino time attendance system with RFID*. Retrieved from https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

4. Instructables. (n.d.). *Arduino + MFRC522 RFID reader*. Retrieved from https://www.instructables.com/Arduino-MFRC522-RFID-READER/

5. Mechatronics System Integration (MCTA3203). (2025). *Week 4: Serial interfacing with microcontroller—Sensors and actuators (ver. 3.0): Designing a motion-activated RFID system*. Universiti Malaysia Sabah.

6. Arduino. (n.d.). *Arduino reference: Serial communication*. Retrieved from https://www.arduino.cc/reference/en/language/functions/communication/serial/

7. Microchip Technology Inc. (n.d.). *ATmega328P datasheet*. Retrieved from https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega328P-Data-Sheet-DS-40002061B.pdf

Appendices

Appedic A-(ARDUINO)

lab3task1.ino

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
 Serial.begin(9600);
 Wire.begin();
 mpu.initialize();

 if (!mpu.testConnection()) {
   Serial.println("MPU6050 connection failed!");
   while (1);
 }

 pinMode(ledPin, OUTPUT);
 digitalWrite(ledPin, LOW); // Start with LED off
}

void loop() {
 // 1. Read motion data
 int16_t ax, ay, az, gx, gy, gz;
 mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

 // 2. Send formatted data to Python (ax,ay,az)
 Serial.print(ax); Serial.print(',');
 Serial.print(ay); Serial.print(',');
 Serial.println(az);
```

```
 // 3. Listen for commands from Python to control the LED
 if (Serial.available() > 0) {
   char cmd = Serial.read();
   if (cmd == 'O') {
     digitalWrite(ledPin, HIGH); // Turn LED ON
   }
   else if (cmd == 'F') {
     digitalWrite(ledPin, LOW);  // Turn LED OFF
   }
 }

 delay(50); // Small delay for stability
}
```

Appendic B-(PYTHON)

```
import serial
import time

ser = serial.Serial( port: 'COM5',  baudrate: 9600, timeout=1)
time.sleep(2)

led_is_on = False # Track the state to avoid flooding commands

try:
    while True:
        if ser.in_waiting > 0: # Fixed from in_available
            line = ser.readline().decode('utf-8').strip()

            if line:
                try:
                    parts = line.split(',')
                    if len(parts) == 3:
                        ax, ay, az = map(int, parts)
                        print(f"X: {ax} | Y: {ay} | Z: {az}")

                        # Check if we need to CHANGE the LED state
                        if abs(ax) <= 10000 and not led_is_on:
                            ser.write(b'O')
                            led_is_on = True
                        elif abs(ax) > 10000 and led_is_on:
                            ser.write(b'F')
                            led_is_on = False

                except ValueError:
                    continue
```

```python
except KeyboardInterrupt:
    print("\nClosing connection...")
    ser.close()
```

```python
import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = "COM5"
BAUD_RATE = 9600
AUTHORIZED_ID = "0013004466"


def detect_circle(rduino):  1 usage
    print("\n[!]Perform circular motion")
    rduino.reset_input_buffer()
    rduino.write(b'R')

    points = []
    start_time = time.time()
    timeout = 3.0

    while time.time() - start_time < timeout:
        if rduino.in_waiting > 0:
            raw = rduino.readline().decode('utf-8', errors='ignore').strip()
            if not raw:
                continue
            parts = raw.split(',')
            if len(parts) >= 3:
                try:
                    ax, ay, az = map(int, parts[:3])
                    points.append((ax, ay, az))
                except ValueError:
                    continue
```

```python

    if len(points) < 10:
        print("Error: Insufficient data.")
        return False

    angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]
    unwrapped = [angles[0]]
    for a in angles[1:]:
        prev, diff = unwrapped[-1], a - unwrapped[-1]
        if diff > 180:
            diff -= 360
        elif diff < -180:
            diff += 360
        unwrapped.append(prev + diff)

    net_rotation = unwrapped[-1] - unwrapped[0]
    rotation_range = max(unwrapped) - min(unwrapped)

    if abs(net_rotation) > 120 or rotation_range > 160:
        return True
    return False


def main():  1 usage
    # We define rduino outside the try so the finally block can access it
    rduino = None
    try:
        rduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2)
```

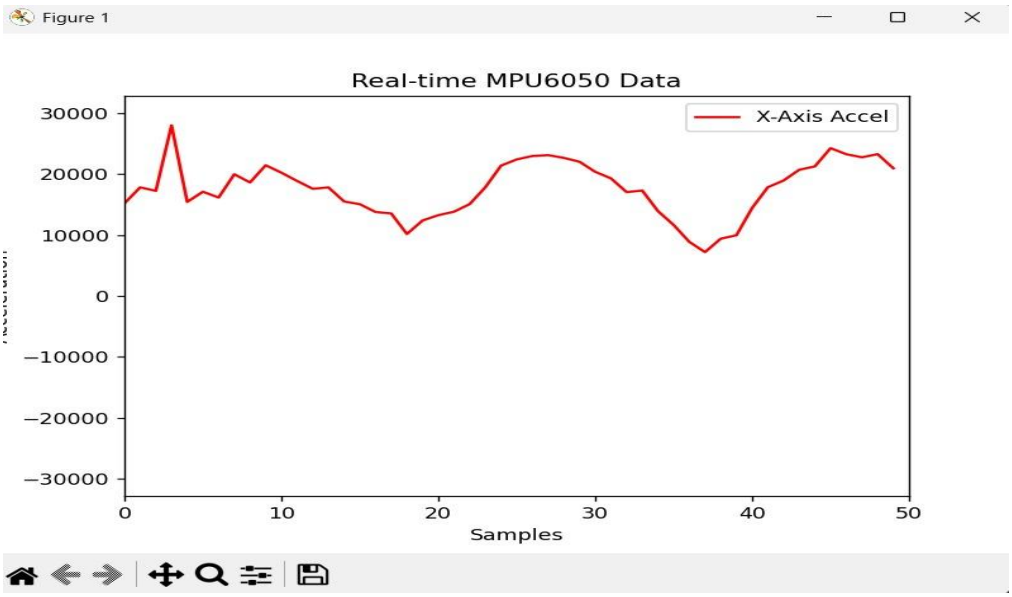
```python
        print("System Initialized. Awaiting RFID Tap...")
        armed = False

        while True:
            card = input("\nTap card: ").strip()
            if card == AUTHORIZED_ID:
                armed = not armed
                if armed:
                    print("Access Pending: Please verify with gesture.")
                    if detect_circle(rduino):
                        print(">>> ACCESS GRANTED: System Unlocked.")
                        rduino.write(b'1')
                    else:
                        print(">>> ACCESS DENIED: Identity failed.")
                        armed = False
                        rduino.write(b'0')
                else:
                    print(">>> SYSTEM SECURED: Locked.")
                    rduino.write(b'0')  # FIXED: Added b'0' here
            else:
                print(">>> INVALID CARD.")
                rduino.write(b'0')

    except Exception as e:
        print(f"An error occurred: {e}")
    except KeyboardInterrupt:
        print("\nShutdown.")
    finally:
        # This block MUST exist to satisfy the 'try' block
        if rduino and rduino.is_open:
```

Appendic C-(Result and Data)

```
X: 17308 | Y: -112 | Z: -1660
X: 17184 | Y: -96 | Z: -1556
X: 17132 | Y: -104 | Z: -1524
X: 17196 | Y: -148 | Z: -1584
```



Test No	RFID Status	RFID Detection	Motion Detection	Servo Angle (°)	LED Status	Access Result
1	0013004466	Authorized	Circular	90	Yellow ON Red OFF	Success
2	0013004466	Authorized	Uncircular	0	Yellow OFF Red OFF	Denied
3	1122334455	Unauthorized	Circular	0	Yellow OFF Red OFF	Denied
4	1128334455	Unauthorized	Uncircular	0	Yellow OFF, red on	Denied

Acknowledgments

We would like to express my sincere gratitude to my lecturer and lab instructor for their continuous guidance, clear explanations, and assistance with experimental setups throughout this experiment. Their detailed instructions with the several needed files attached have helped us understand how MPU6050 sensor works and also setup RFID reader .Therefore,from that we are able to learn how to implement multi-layered security, such as Two-Factor authentication.

I would also like to thank the teaching assistants for their patience and assistance during the lab session. Their help in identifying wiring mistakes and explaining how to correct the logic in the program was very valuable.

In addition, I truly appreciate my classmates and lab partners for their teamwork, cooperation, and ideas during the experiment. We worked together to troubleshoot connection issues, share components, and test the code to make sure everything functioned as expected.

Overall, the success of this experiment would not have been possible without the guidance and support from both the instructors and my lab partners.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.Therefore, we agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: MOHAMED ZAWIR FARIS BIN MOHAMED FAUZI

Matric Number: 2329493

Understand /

Agree /

Signature:

Name: MUHAMAD HARITH HAIKAL BIN ZULKIFLI

Matric Number: 2319963

Read /

Understand /

Agree /

Signature:

Name: MUHAMMAD AIDIL AIMAN BIN WAKIL @ MUHAMAD SYUKRI

Matric Number: 2316967

Read /

Understand /

Agree /